

MARAN PMS — Prompt 57: Tablero Operativo del Día

Objetivo

Una pantalla que le da al recepcionista y al gerente una foto completa del día en segundos. Todo en un solo lugar, sin tener que ir módulo por módulo.

Ubicación

Nueva ruta `/operaciones` accesible desde el sidebar, entre Dashboard y Planning.

Archivos a crear/modificar

- `client/src/pages/operaciones.tsx` — página nueva
 - `client/src/App.tsx` — agregar ruta
 - `client/src/components/app-sidebar.tsx` — agregar ítem en menú
 - `server/routes.ts` — 1 endpoint nuevo `/api/operaciones/resumen`
-

PARTE 1 — Endpoint backend (server/routes.ts)

Agregar junto a los endpoints de dashboard:

```
// ===== TABLERO OPERATIVO =====

app.get("/api/operaciones/resumen", requireAuth, async (req, res) => {
  try {
    const today = new Date().toLocaleDateString("en-CA", {
      timeZone: "America/Argentina/Buenos_Aires"
    });

    // 1. Check-ins del día (confirmadas/pendientes con checkInDate = hoy)
    const checkInsHoy = await db
      .select({
        id: reservations.id,
```

```

        reservationCode: reservations.reservationCode,
        checkInDate: reservations.checkInDate,
        checkOutDate: reservations.checkOutDate,
        status: reservations.status,
        roomId: reservations.roomId,
        guestId: reservations.guestId,
    })
    .from(reservations)
    .where(
        and(
            eq(reservations.checkInDate, today),
            inArray(reservations.status, ["confirmed", "pending", "tentative"] as a
        )
    );

// 2. Check-outs del día (checked_in con checkOutDate = hoy)
const checkOutsHoy = await db
    .select({
        id: reservations.id,
        reservationCode: reservations.reservationCode,
        checkInDate: reservations.checkInDate,
        checkOutDate: reservations.checkOutDate,
        status: reservations.status,
        roomId: reservations.roomId,
        guestId: reservations.guestId,
    })
    .from(reservations)
    .where(
        and(
            eq(reservations.checkOutDate, today),
            eq(reservations.status, "checked_in")
        )
    );

// 3. Reservas in-house con saldo pendiente
const inHouseReservations = await db
    .select()
    .from(reservations)
    .where(eq(reservations.status, "checked_in"));

const foliosConSaldo: any[] = [];

for (const res of inHouseReservations) {
    const [chargesSum] = await db
        .select({ total: sql<number>`COALESCE(SUM(${charges.amount}::numeric), 0)`
        .from(charges)
        .where(and(eq(charges.reservationId, res.id), eq(charges.status, "active"

```

```

const [paymentsSum] = await db
  .select({ total: sql<number>`COALESCE(SUM(${payments.amount}::numeric), 0
  .from(payments)
  .where(and(eq(payments.reservationId, res.id), eq(payments.status, "activ

const balance = Number(chargesSum?.total ?? 0) - Number(paymentsSum?.total

if (balance > 0.01) {
  foliosConSaldo.push({
    reservationId: res.id,
    reservationCode: res.reservationCode,
    roomId: res.roomId,
    guestId: res.guestId,
    balance: Math.round(balance * 100) / 100,
  });
}
}

// 4. Cajas abiertas ahora
const cajasAbiertas = await db
  .select({
    id: cashShifts.id,
    area: cashShifts.area,
    shiftNumber: cashShifts.shiftNumber,
    openedBy: cashShifts.openedBy,
    openedAt: cashShifts.openedAt,
    autoCreado: cashShifts.autoCreado,
  })
  .from(cashShifts)
  .where(eq(cashShifts.status, "open"));

// 5. Habitaciones sucias (estado dirty)
const habitacionesSuccias = await db
  .select({ id: rooms.id, roomNumber: rooms.roomNumber, floor: rooms.floor })
  .from(rooms)
  .where(eq(rooms.status, "dirty"));

// 6. Tareas de housekeeping pendientes hoy
const tareasHoy = await db
  .select({ id: housekeepingTasks.id, status: housekeepingTasks.status })
  .from(housekeepingTasks)
  .where(eq(housekeepingTasks.scheduledDate, today));

const tareasPendientes = tareasHoy.filter(t => t.status === "pending").length
const tareasEnProceso = tareasHoy.filter(t => t.status === "in_progress").len
const tareasCompletadas = tareasHoy.filter(t => t.status === "completed" || t

```

```

// 7. Incidencias abiertas (si existe la tabla)
let incidenciasAbiertas = 0;
let incidenciasCriticas = 0;
try {
  const incidents = await db
    .select({ severity: systemIncidents.severity, status: systemIncidents.status })
    .from(systemIncidents)
    .where(
      inArray(systemIncidents.status, ["pendiente", "en_revision"] as any)
    );
  incidenciasAbiertas = incidents.length;
  incidenciasCriticas = incidents.filter(i => i.severity === "critica").length;
} catch {
  // tabla puede no existir aún
}

// 8. Resumen de cajas del día
const movimientosHoy = await db
  .select({
    area: cashMovements.area,
    amount: cashMovements.amount,
    movementType: cashMovements.movementType,
    anulado: cashMovements.anulado,
  })
  .from(cashMovements)
  .where(
    and(
      sql`DATE(${cashMovements.createdAt} AT TIME ZONE 'America/Argentina/Buenos_Aires') = DATE(${today})`,
      eq(cashMovements.anulado, false)
    )
  );

const recaudacionPorArea: Record<string, number> = {};
for (const mov of movimientosHoy) {
  if (!recaudacionPorArea[mov.area]) recaudacionPorArea[mov.area] = 0;
  const amt = parseFloat(mov.amount);
  recaudacionPorArea[mov.area] += mov.movementType === "income" ? amt : -amt;
}

const totalRecaudado = Object.values(recaudacionPorArea).reduce((s, v) => s + v);

res.json({
  fecha: today,
  checkIns: {
    total: checkInsHoy.length,
    reservas: checkInsHoy,
  },
  totalRecaudado,
});

```

```

    },
    checkOuts: {
      total: checkOutsHoy.length,
      reservas: checkOutsHoy,
    },
    foliosConSaldo: {
      total: foliosConSaldo.length,
      items: foliosConSaldo,
    },
    cajas: {
      abiertas: cajasAbiertas.length,
      detalle: cajasAbiertas,
      recaudacionPorArea,
      totalRecaudado: Math.round(totalRecaudado * 100) / 100,
    },
    housekeeping: {
      habitacionesSuccias: habitacionesSuccias.length,
      tareasPendientes,
      tareasEnProceso,
      tareasCompletadas,
      totalTareas: tareasHoy.length,
    },
    incidencias: {
      abiertas: incidenciasAbiertas,
      criticas: incidenciasCriticas,
    },
  });

} catch (error) {
  console.error("Error en tablero operativo:", error);
  res.status(500).json({ error: "Error generando resumen operativo" });
}
});

```

Imports necesarios en routes.ts (verificar que estén):

```

import {
  reservations, rooms, charges, payments,
  cashShifts, cashMovements, housekeepingTasks,
  systemIncidents, // si ya existe
} from "@shared/schema";

```

PARTE 2 — Página frontend

(client/src/pages/operaciones.tsx)

Crear archivo nuevo:

```
import { useQuery } from "@tanstack/react-query";
import { Card, CardContent, CardHeader, CardTitle } from "@components/ui/card";
import { Badge } from "@components/ui/badge";
import { Skeleton } from "@components/ui/skeleton";
import {
  LogIn, LogOut, CreditCard, Building2,
  CheckCircle, AlertTriangle, RefreshCw,
  Wallet, ClipboardCheck, Clock
} from "lucide-react";
import { Button } from "@components/ui/button";
import { useLink } from "wouter";

const AREA_LABEL: Record<string, string> = {
  reception: "Recepción",
  restaurant: "Restaurante",
  spa: "SPA",
  events: "Eventos",
};

function formatCurrency(n: number) {
  return "$ " + Math.round(n).toLocaleString("es-AR");
}

function formatTime(dateStr: string) {
  return new Date(dateStr).toLocaleTimeString("es-AR", {
    hour: "2-digit", minute: "2-digit", timeZone: "America/Argentina/Buenos_Aires"
  });
}

export default function OperacionesPage() {
  const { data, isLoading, refetch, isFetching } = useQuery<any>({
    queryKey: ["/api/operaciones/resumen"],
    queryFn: async () => {
      const res = await fetch("/api/operaciones/resumen", { credentials: "include" });
      if (!res.ok) throw new Error("Error cargando tablero");
      return res.json();
    },
    refetchInterval: 60_000, // auto-refresh cada 60 segundos
  });

  const today = new Date().toLocaleDateString("es-AR", {
    weekday: "long", day: "2-digit", month: "long",
  });
```

```

    timeZone: "America/Argentina/Buenos_Aires"
  });

return (
  <div className="p-6 space-y-6">

    {/* Header */}
    <div className="flex items-center justify-between">
      <div>
        <h1 className="text-2xl font-bold capitalize">{today}</h1>
        <p className="text-muted-foreground text-sm">Tablero operativo del día<
      </div>
      <Button
        variant="outline"
        size="sm"
        onClick={() => refetch()}
        disabled={isFetching}
        data-testid="btn-refresh-tablero"
      >
        <RefreshCw className={ `h-4 w-4 mr-2 ${isFetching ? "animate-spin" : ""}
        Actualizar
      </Button>
    </div>

    {isLoading ? (
      <div className="grid grid-cols-2 md:grid-cols-4 gap-4">
        {Array(8).fill(0).map((_, i) => <Skeleton key={i} className="h-28" />)}
      </div>
    ) : (
      <>
        {/* Fila 1 - Métricas principales */}
        <div className="grid grid-cols-2 md:grid-cols-4 gap-4">

          {/* Check-ins */}
          <Card className="border-l-4 border-l-green-400 hover:shadow-md transi
            onClick={() => window.location.href = "/check-in"}>
            <CardContent className="p-4">
              <div className="flex items-center gap-3">
                <div className="p-2 rounded-md bg-green-100 dark:bg-green-900/3
                  <LogIn className="h-5 w-5 text-green-600 dark:text-green-400"
                </div>
                <div>
                  <p className="text-3xl font-bold">{data?.checkIns?.total ?? 0
                  <p className="text-xs text-muted-foreground">Check-ins hoy</p>
                </div>
              </div>
            </CardContent>

```

```
</Card>
```

```
{/* Check-outs */}
```

```
<Card className="border-l-4 border-l-amber-400 hover:shadow-md transitionClick={() => window.location.href = "/check-out"}>
```

```
<CardContent className="p-4">
```

```
<div className="flex items-center gap-3">
```

```
<div className="p-2 rounded-md bg-amber-100 dark:bg-amber-900/3">
```

```
<LogOut className="h-5 w-5 text-amber-600 dark:text-amber-400">
```

```
</div>
```

```
<div>
```

```
<p className="text-3xl font-bold">{data?.checkOuts?.total ??
```

```
<p className="text-xs text-muted-foreground">Check-outs hoy</
```

```
</div>
```

```
</div>
```

```
</CardContent>
```

```
</Card>
```

```
{/* Folios con saldo */}
```

```
<Card className={`border-l-4 ${data?.foliosConSaldo?.total ?? 0} > 0`>
```

```
<CardContent className="p-4">
```

```
<div className="flex items-center gap-3">
```

```
<div className={`p-2 rounded-md ${data?.foliosConSaldo?.total`>
```

```
<CreditCard className={`h-5 w-5 ${data?.foliosConSaldo?.total`>
```

```
</div>
```

```
<div>
```

```
<p className="text-3xl font-bold">{data?.foliosConSaldo?.total`>
```

```
<p className="text-xs text-muted-foreground">Saludos pendiente
```

```
</div>
```

```
</div>
```

```
</CardContent>
```

```
</Card>
```

```
{/* Habitaciones sucias */}
```

```
<Card className={`border-l-4 ${data?.housekeeping?.habitacionesSucias`>
```

```
<CardContent className="p-4">
```

```
<div className="flex items-center gap-3">
```

```
<div className={`p-2 rounded-md ${data?.housekeeping?.habitaciones`>
```

```
<Building2 className={`h-5 w-5 ${data?.housekeeping?.habitaciones`>
```

```
</div>
```

```
<div>
```

```
<p className="text-3xl font-bold">{data?.housekeeping?.habitaciones`>
```

```
<p className="text-xs text-muted-foreground">Hab. para limpiar
```

```
</div>
```

```
</div>
```

```
</CardContent>
```

```
</Card>
```

```
</div>
```

```
{/* Fila 2 – Cajas + Housekeeping + Incidencias */}
```

```
<div className="grid grid-cols-1 md:grid-cols-3 gap-4">
```

```
  {/* Cajas */}
```

```
  <Card>
```

```
    <CardHeader className="pb-3">
```

```
      <CardTitle className="text-sm flex items-center gap-2">
```

```
        <Wallet className="h-4 w-4" />
```

```
        Cajas del día
```

```
      </CardTitle>
```

```
    </CardHeader>
```

```
    <CardContent className="space-y-3">
```

```
      {/* Recaudación por área */}
```

```
      {Object.entries(data?.cajas?.recaudacionPorArea ?? {}).map(([area
```

```
        <div key={area} className="flex items-center justify-between te
```

```
        <div className="flex items-center gap-2">
```

```
          <div className={`w-2 h-2 rounded-full ${
```

```
            area === "reception" ? "bg-blue-400" :
```

```
            area === "restaurant" ? "bg-orange-400" :
```

```
            area === "spa" ? "bg-purple-400" : "bg-green-400"
```

```
          ]} />
```

```
          <span className="text-muted-foreground">{AREA_LABEL[area] ?
```

```
        </div>
```

```
        <span className="font-medium">{formatCurrency(total)}</span>
```

```
      </div>
```

```
    )))
```

```
    <div className="border-t pt-2 flex justify-between text-sm font-b
```

```
      <span>Total recaudado</span>
```

```
      <span>{formatCurrency(data?.cajas?.totalRecaudado ?? 0)}</span>
```

```
    </div>
```

```
  {/* Cajas abiertas */}
```

```
  <div className="border-t pt-2 space-y-1">
```

```
    <p className="text-xs text-muted-foreground">Cajas abiertas aho
```

```
    {(data?.cajas?.detalle ?? []).length === 0 ? (
```

```
      <p className="text-xs text-muted-foreground italic">Ninguna</
```

```
    ) : (
```

```
      (data?.cajas?.detalle ?? []).map((c: any) => (
```

```
        <div key={c.id} className="flex items-center justify-betwee
```

```
        <div className="flex items-center gap-1.5">
```

```
          <div className="w-1.5 h-1.5 rounded-full bg-green-500 a
```

```
          <span className="text-xs">{AREA_LABEL[c.area] ?? c.area
```

```
            {c.autoCreado && (
```

```
              <Badge variant="outline" className="text-[10px] px-1
```

```
            )}
```

```

        </div>
        <span className="text-xs text-muted-foreground">
            {c.openedBy ?? "-"} · {formatTime(c.openedAt)}
        </span>
    </div>
    ))
    })
</div>
</CardContent>
</Card>

{/* Housekeeping */}
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-sm flex items-center gap-2">
            <ClipboardCheck className="h-4 w-4" />
            Housekeeping
        </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        {/* Barra de progreso */}
        {(data?.housekeeping?.totalTareas ?? 0) > 0 && (
            <div className="space-y-1">
                <div className="flex justify-between text-xs text-muted-foreg">
                    <span>Progreso del día</span>
                    <span>
                        {data?.housekeeping?.tarefasCompletadas ?? 0} / {data?.hou
                    </span>
                </div>
                <div className="h-2 bg-muted rounded-full overflow-hidden">
                    <div
                        className="h-full bg-green-500 transition-all"
                        style={{
                            width: `${Math.round(
                                ((data?.housekeeping?.tarefasCompletadas ?? 0) /
                                Math.max(data?.housekeeping?.totalTareas ?? 1, 1)) *
                            )}%`
                        }}
                    </div>
                </div>
            </div>
        ))

        <div className="space-y-2">
            {[
                { label: "Pendientes", value: data?.housekeeping?.tarefasPendi
                { label: "En proceso", value: data?.housekeeping?.tarefasEnPro

```

```

        { label: "Completadas", value: data?.housekeeping?.tareasComp
].map(({ label, value, color }) => (
    <div key={label} className="flex justify-between text-sm">
        <span className="text-muted-foreground">{label}</span>
        <span className={`font-medium ${color}`}>{value ?? 0}</span>
    </div>
    )))
</div>

<div className="border-t pt-2 flex justify-between text-sm">
    <span className="text-muted-foreground">Hab. sucias sin tarea</span>
    <span className={`font-medium ${data?.housekeeping?.habitacion
{data?.housekeeping?.habitacionesSuccias ?? 0}
    </span>
</div>
</CardContent>
</Card>

{/* Incidencias */}
<Card>
    <CardHeader className="pb-3">
        <CardTitle className="text-sm flex items-center gap-2">
            <AlertTriangle className="h-4 w-4" />
            Incidencias abiertas
        </CardTitle>
    </CardHeader>
    <CardContent className="space-y-3">
        {(data?.incidencias?.abiertas ?? 0) === 0 ? (
            <div className="flex items-center gap-2 text-green-600">
                <CheckCircle className="h-4 w-4" />
                <span className="text-sm">Sin incidencias activas</span>
            </div>
        ) : (
            <>
                <div className="flex items-center gap-3">
                    <div className="text-3xl font-bold text-orange-600">
                        {data?.incidencias?.abiertas}
                    </div>
                    <div className="text-sm text-muted-foreground">
                        {data?.incidencias?.abiertas === 1 ? "incidencia abierta"
                    </div>
                </div>
            </div>
            {(data?.incidencias?.criticas ?? 0) > 0 && (
                <div className="flex items-center gap-2 p-2 bg-red-50 dark:
                    <AlertTriangle className="h-4 w-4 text-red-600" />
                    <span className="text-sm text-red-600 font-medium">
                        {data?.incidencias?.criticas} crítica{data?.incidencias

```

```

        </span>
      </div>
    )}
  </>
)}
<Button
  variant="outline"
  size="sm"
  className="w-full"
  onClick={() => window.location.href = "/administration"}
>
  Ver bitácora
</Button>
</CardContent>
</Card>

</div>

{/* Fila 3 – Folios con saldo (si hay) */}
{(data?.foliosConSaldo?.items ?? []).length > 0 && (
  <Card>
    <CardHeader className="pb-3">
      <CardTitle className="text-sm flex items-center gap-2">
        <CreditCard className="h-4 w-4 text-red-500" />
        Huéspedes in-house con saldo pendiente
      </CardTitle>
    </CardHeader>
    <CardContent>
      <div className="space-y-2">
        {data.foliosConSaldo.items.map((folio: any) => (
          <div
            key={folio.reservationId}
            className="flex items-center justify-between p-2 rounded-md"
            onClick={() => window.location.href = `/reservations`}
          >
            <div className="flex items-center gap-3">
              <Badge variant="outline" className="text-xs font-mono">
                {folio.reservationCode}
              </Badge>
            </div>
            <span className="text-sm font-bold text-red-600">
              {formatCurrency(folio.balance)}
            </span>
          </div>
        ))}
      </div>
    </CardContent>
  </Card>
)}

```

```
        </Card>
      )}

    </>
  )}
</div>
);
}
```

PARTE 3 — Registrar la ruta (client/src/App.tsx)

```
// Agregar import:
import OperacionesPage from "@pages/operaciones";

// Agregar ruta (entre Dashboard y Planning):
<Route path="/operaciones" component={OperacionesPage} />
```

PARTE 4 — Agregar al sidebar (client/src/components/app-sidebar.tsx)

Buscar donde están los ítems de Dashboard y Planning, agregar entre ellos:

```
{
  title: "Operaciones",
  url: "/operaciones",
  icon: LayoutDashboard, // o Activity, o Gauge
}
```

NOTAS TÉCNICAS

- **Auto-refresh cada 60 segundos** — la página se actualiza sola, ideal para tenerla abierta en recepción
- **Sin tabla nueva** — todo se calcula en tiempo real desde las tablas existentes
- **El cálculo de saldo** hace queries individuales por reserva in-house — si hay muchas reservas simultáneas considerar optimizar con una query agregada

- `systemIncidents` está en `try/catch` porque la tabla puede no existir todavía — una vez implementado el Prompt 56 va a funcionar automáticamente
- **Los links** usan `window.location.href` por simplicidad — si se prefiere usar el router de wouter se puede cambiar a `useLocation` de wouter
- **Timezone** — usar siempre `America/Argentina/Buenos_Aires` en el backend para calcular “hoy”
- El endpoint `/api/operaciones/resumen` no requiere parámetros — siempre devuelve el día actual